

Aura: Process Overview

How a one-person product team and an AI implementer build an MMO together

Point-in-time snapshot, 2026-08-12. Written for an outside reader who wants to learn from or improve the process. The game itself (a browser MMO called Aura) appears only as illustration.

Contents

1. The process in one page
2. The big picture: where truth lives
3. Anatomy of one work item
4. The process over time
5. What changed from the initial plan
6. What to borrow, and where it strains

1. The process in one page

The team is one human and one AI. The human acts as product owner (PO): they make every design decision, play every build, and never write code. The implementer is an AI coding agent (Claude Code) that works in discrete sessions with no built-in memory between them. A second human occasionally pushes to the repository, but the core loop is PO plus agent.

Every distinctive feature of the process follows from one fact: the implementer forgets everything between sessions and can be confidently wrong within one. So the process is built around externalized memory, decision records that can be re-found, work sliced to fit a single session, and a verification stack that assumes the implementer's claims need proof.

The loop for any piece of work:

1. **Planning session.** One work item is designed in conversation and written up as a plan document: what changes and why, a chunk breakdown, numbered PO decisions, known landmines, open questions, and a test strategy. No production code is written.
2. **Execution sessions.** Each chunk of the plan is built in its own session, following the plan. A chunk ends with a fixed verification tail: tests written to fail first, mutation checks, the full test suite, a headless in-game smoke run, and an explicit statement of database-schema impact (even when it is "none").
3. **PO acceptance.** The PO plays the change in the real game. Automated green is necessary but never sufficient; the in-game pass is the actual acceptance gate.
4. **Bookkeeping.** The chunk's outcome, decisions, and surprises are written back into the plan's ledger and the project status file. When a plan's last chunk lands, the whole

document moves to an archive folder and stays there as the permanent rationale record.

Status lives in exactly four places with non-overlapping jobs: a *current state* file read at the start of every session, a *sequence* file holding the decided build order, *per-item ledgers* inside each plan document, and a *cross-session index* so a fresh session can find the other three. Nothing is duplicated between the layers; each question has one authoritative home.

Two standing rules keep the human in control: the agent never commits, branches, or pushes on its own, and every concrete number in the game is a placeholder until the PO explicitly marks it final.

2. The big picture: where truth lives

2.1 The four-layer status model

The central problem is that the implementer starts every session blank. The answer is a small set of documents with strictly separated jobs:

Layer	Job	Discipline that keeps it useful
<code>CLAUDE.md</code> <code>\$Status</code>	The authoritative <i>current</i> state: last completed chunk, what is next, open PO calls, standing locks and gotchas. Loaded automatically into every session.	Hard-capped at the last completed entry plus two prior. Older entries move verbatim to a history file, so the section never silts up.
<code>roadmap.md</code> "Execution order"	The authoritative <i>sequence</i> and per-step outcome. Steps are sealed here when they complete.	Explicitly not a session log; the item numbering is an enumeration, the decided order is a separate section.
<code>plan-*.md</code> ledgers	The per-work-item record: every decision with its rationale, every chunk's outcome and commit hash, every landmine found while building.	One plan document per work item; superseded decisions stay in the document, marked as superseded, so history is never rewritten.
<code>MEMORY.md</code> index	Cross-session index: one line per remembered fact, pointing at a small file holding the detail.	Holds only what the other layers do not record (working preferences, hard-won gotchas), never project status.

The rule "don't duplicate between layers" is load-bearing. When the same fact lives in two places, one copy goes stale and a future session cannot tell which one; the layer model makes staleness structurally detectable, because each fact has one home.

2.2 Live versus archived

The docs folder is split in two: `docs/` holds anything with something still open; `docs/archive/` holds finished work. A plan moves to the archive the moment its last chunk lands. Archived plans are not dead: they are frequently the only design record for a system that is very much alive, so they are read for *why*, never for *current state*. The split's real value is attention: a session scanning the live folder sees only what can still be acted on.

2.3 The backlog and the memory

Unscoped ideas live in a numbered backlog (§1, §2, ...) so that decisions can cite them stably ("blocked on §39"). Separately, the agent maintains its own memory files across sessions: one file per durable fact, indexed by one line each. The convention is that memory holds only what the repository cannot derive, mostly process lessons, for example: "a content edit does not invalidate the Go test cache, always rerun with `-count=1`", or "never delete live database rows while the server is running".

2.4 Repeated procedures become skills

Recurring rituals are packaged as reusable, versioned instruction sets ("skills") the agent invokes by name: `chunk-wrap` encodes the session-end bookkeeping, `verify` the headless in-game smoke test, `add-content` the landmine checklist for authoring game content, `playtest` the "get it running so the PO can click into it" handover. This is the same move as turning a wiki runbook into a script: the procedure stops depending on anyone remembering it.

3. Anatomy of one work item

3.1 The plan document

A planning session produces one document per work item, and the document has a stable internal vocabulary:

- **Numbered decisions (D1, D2, ...):** every PO ruling gets a number, the rationale, and the alternatives considered. Later sessions cite decisions by number. When a decision is reversed, the reversal is recorded next to the original, never by editing it away.
- **Numbered landmines (L1, L2, ...):** non-obvious code facts that would break a naive implementation, found by surveying the code *during planning*. Example from the fast-travel plan: the assumption "world visibility and map visibility are one fact" was written into the plan, later proven to be two facts, and the correction cost a chunk. The lesson was recorded as a general rule: "two channels carry the same fact" is a claim to verify, not to assume.
- **Open questions:** anything the PO has not ruled on yet, kept visibly separate from decisions.

- **A chunk breakdown:** the work sliced into sessions.

ILLUSTRATION: PRICING OPTIONS AGAINST CODE, NOT ARGUMENT

In one plan, the recommendation was to reuse an existing wire payload rather than add a new one, because reuse looked cheaper. Pricing both options against the actual code showed that every option needed a payload anyway, so "reuse" merely added a second timer to an existing structure. The recommendation was overturned by measurement before any code was written. The process treats this as the planning session's core job: decisions are made against surveyed code, not against plausibility.

3.2 Chunking

Chunks are sized to one session and ordered so that each leaves the game shippable. A recurring pattern is the **plumbing/content split**: chunk "a" restructures code with deliberately zero behaviour change (and is verified by proving nothing changed), then chunk "b" ships the visible change on top. This separates "did the refactor break anything" from "is the new behaviour right", which are otherwise entangled in one diff.

3.3 The verification tail

Every chunk ends with the same stack, cheapest first:

1. **Red-first tests.** New tests are shown to fail before the change makes them pass (test-driven development, enforced as ritual rather than preference).
2. **Mutation checks.** The implementer deliberately breaks the new code in N ways and confirms the tests catch all N. Chunk reports state this as a count ("11/11 mutations caught"). A surviving mutation is treated as a finding: sometimes a missing test, and at least once it exposed dead code instead.
3. **Full suite, cache bypassed.** The whole test suite with the test cache disabled, plus the race detector. The cache bypass exists because a measured incident showed content edits do not invalidate Go's test cache, so a stale green can hide a real break.
4. **Boot check.** The server must boot with zero warnings and zero errors; content validation is pushed into boot so a typo becomes a refusal to start rather than a silent misbehaviour.
5. **Headless in-game smoke.** Scripted browser harnesses drive the real client against the real server. Known-flaky harness legs are recorded *with their baseline pass rates*, because misreading an inherited flake as your own regression once cost two chunks of false diagnosis. The standing rule: measure the failure rate before diagnosing a flake.
6. **Schema impact statement.** Every chunk explicitly states its database and wire-protocol impact, even when the answer is "none at every layer". Absence of impact is a recorded finding, not an omission.

Then the PO plays it. The distinction matters because harnesses have gone green while a bug shipped, twice in one plan; the fix was to make the harness assert the thing that lied

(render depth as stage indices), but the deeper rule stayed: automation narrows the space, the human walk closes it.

3.4 The wrap

Session-end bookkeeping is itself proceduralized (the `chunk-wrap` skill), tiered by session size: a full plan chunk gets the complete ritual (ledger entry, status update, memory update, archive move if the plan is done); a small bugfix gets a light wrap; a docs-only session even less. Commits happen only when the PO asks.

4. The process over time

The project began in early July 2026 as a fork of an existing open-source survival game, chosen for its working multiplayer foundation. The process was not designed up front; nearly every element above hardened out of a specific incident.

When	What happened	What it added to the process
Early July	Survival systems stripped, single-resource model established, skill system migrated in phases.	Block-shaped work with archive records; the habit of recording <i>why</i> alongside <i>what</i> .
2026-07-08	The build order was decided explicitly: systems first, content once against finished systems, productionize last.	The sequence became a document ("Execution order") instead of an intention. Its trade-off (no real fun feedback until the content pass) was written down as accepted, with mitigations named.
Mid July	World editor, mob depth, atmosphere, skill vocabulary, NPC teaching: one plan document each, all chunked, all PO-verified in-game.	The plan-doc anatomy stabilized (numbered decisions, landmines, chunk ledgers). A simulation harness was built as a gate before content, so balancing had an instrument.
2026-07-21	First live deploy to a rented server; first external playtest the next day.	A standing playtest-intake document: feedback rounds are appended, triaged into passes, and picked from deliberately. Feedback became a queue, not interrupts.
Late July	Rebrand and cleanup; entity-model convergence; quest system.	The archive folder was created, splitting live from finished docs. Plan docs started recording reframes: the entity plan is kept because the code contradicted the task as stated, and the survey found the real defect.

Early August	Accounts and persistence (the largest step, seven chunks over a week), then the character-sacrifice loop as its first consumer.	Schema-impact statements became mandatory on every chunk. Plain-language HTML summaries per chunk appeared for human stakeholders. Session handover notes were made explicitly disposable: deleted when their chunk lands.
2026-08-03	The status section had grown into a scroll.	The cap: last completed plus two prior, overflow archived verbatim to a history file. Bookkeeping itself became a skill.
Ongoing	Incidents keep hardening the tail.	Each measured failure becomes a standing rule: the test-cache bypass, the flake-rate rule, "never clean live database rows under a running server", "a harness that needs editing when a value is retuned has stopped being evidence".

The visible trend: the process accretes *rules with provenance*. Almost every rule in the project's instruction files cites the incident that created it, often with the commit hash. That keeps the rule set honest; a rule that cannot name its incident is a candidate for deletion.

5. What changed from the initial plan

The initial roadmap survives, but several of its assumptions did not. The changes fall into four patterns, and the patterns say more about the process than the changes do.

5.1 Scope moved in both directions, by explicit ruling

- **Cut:** line-of-sight for auras was removed from scope entirely (auras now pass through walls, a flag already built was deleted). The day/night cycle shipped, broke avatar visibility, and was switched off rather than fixed, with the re-enable condition written down.
- **Pulled in:** the character-sacrifice loop was moved *into* v1 by a PO ruling mid-project, because persistence needed a first consumer to prove itself against.
- **Cut mid-plan:** one fast-travel chunk was cancelled after a feel pass showed the feature read well without it. Applying "you aren't gonna need it" to an approved plan document, not just to code, is treated as a feature of the process.

5.2 Play falsified design, quickly and cheaply

The XP formula plan tuned a reward band to a requirement that had been recorded weeks earlier. Twenty minutes of the PO actually playing falsified it, and the reversal shipped the same day. The process lesson was recorded: a written requirement is a hypothesis until played, and the in-game boundary the PO reported independently matched the reference game's value to the digit.

5.3 Surveys were wrong, and being wrong was budgeted for

Plans record their own survey errors instead of hiding them: one plan claimed "no schema change" and "no fog of war" and both were reversed mid-plan; another claimed other players already appeared on the minimap, which was simply false, and the correction made the chunk smaller. Because the survey is written down before building, a wrong survey is a diff against the plan, visible and attributable, rather than silent drift.

5.4 The placeholder discipline

From the start, every concrete number (drop rates, radii, costs, caps) is declared a placeholder until the PO marks it final, and even "final" is defined as "first-pass settled, still tunable, just note the change". This single convention prevents the most common failure of design-by-conversation: an example number silently becoming a decision.

6. What to borrow, and where it strains

6.1 Worth borrowing, even without an AI in the loop

- **One home per fact.** The four-layer status model is just aggressive normalization of project state. Any team whose wiki, tracker, and README disagree already pays the cost this eliminates.
- **Decisions as numbered, immutable records.** Reversals sit next to originals. Six weeks in, the project can answer "why is it like this" for essentially any behaviour, with the alternatives that were rejected and the price of each.
- **The plumbing/content chunk split.** Verifying "nothing changed" and "the right thing changed" as two separate deliveries is cheap and catches a whole class of refactor regressions.
- **Mutation counts as a first-class report line.** "Tests pass" claims little; "I broke it 11 ways and the tests caught all 11" claims something falsifiable.
- **Rules with provenance.** Process rules cite their incident. This both justifies the rule and defines when it may die.
- **Explicit impact statements.** Requiring every change to state its schema and wire impact, including "none", turns the most dangerous category of omission into a checklist line.

6.2 Where the current process strains

An outside improver would probably look here first:

- **The status file is dense to the point of hostility.** The current-state section is written in a compressed, glyph-heavy idiom optimized for the agent's re-reading, not for a human newcomer. It works for a team of two; a third contributor would need the onboarding documents, which are point-in-time snapshots and already dated.

- **Verification is strong per chunk, weak across time.** There is no CI: the whole test stack runs when a session runs it. The project's own code-quality review recommends tests in CI as one of three cheap fixes; it remains open. Known-flaky tests are documented rather than owned, and the list is growing.
- **Acceptance serializes on one person.** The PO's in-game pass is the real gate, which is correct for design fidelity but makes the PO the throughput ceiling, and "PO-seen" versus "not yet PO-seen in play" states accumulate as debt.
- **Deliberate fragility is a bet with a timer.** The live database has no backups, by explicit ruling (the server is a testing ground). The ruling is recorded, revisit triggers are named, and the security items are tracked as owed; still, the project now contains a feature whose entire point is permanence, sitting on a database that is declared losable. The process handles this correctly as bookkeeping; whether the bet itself is right is exactly the kind of question an outside reviewer should press.
- **Line references rot.** Plans pin code facts to line numbers at a specific commit; later work moves them. The convention (pin to a hash, flag staleness in the banner) contains the damage but does not remove the re-verification tax.

The one-sentence summary: the process compensates for a memoryless, fallible implementer with externalized state, decision records that never get rewritten, session-sized work slices, and a verification stack that treats every claim as unproven until a test fails first, a mutation dies, and a human has played the build. Most of it would improve an all-human team too; the parts under most strain are the ones that assume the team stays this small.